Module 1: Foundation in analyzing algorithms

Lesson 1: Algorithms as Technology

Learning Outcomes

- LO 1.1.1 Explain the importance and practical applications of algorithms.
- LO 1.1.2 Explore mathematical tools for needed for algorithm analysis.

Algorithms as Technology

Informally, an ***algorithm*** is any well-defined computational procedure that takes some value, or set of values, as ***input*** and produces some value, or set of values, as ***output***. An algorithm is thus a sequence of computational steps that transform the input into the output.

We can also view an algorithm as a tool for solving a well-specified ***computational problem***. The statement of the problem specifies in general terms the desired input/output relationship. The algorithm describes a specific computational procedure for achieving that input/output relationship.

For example, we might need to sort a sequence of numbers into non-decreasing order.

Input: A sequence of n numbers (a1, a2, a3,…an).
Output: A permutation (reordering) (a1'≤, a2'≤, a3'≤,…an).

Such an input sequence is called an ***instance*** of the sorting problem. In general, an ***instance of a problem*** consists of the input (satisfying whatever constraints are imposed in the problem statement) needed to compute a solution to the problem.

Because many programs use it as an intermediate step, sorting is a fundamental operation in computer science. As a result, we have a large number of good sorting algorithms at our disposal. Which algorithm is best for a given application depends on—among other factors—the number of items to be sorted, the extent to which the items are already somewhat sorted, possible restrictions on the item values, the architecture of the computer, and the kind of storage devices to be used: main memory, disks, or even tapes.

An algorithm is said to be correct if, for every input instance, it halts with the correct output. We say that a correct algorithm solves the given computational problem. An incorrect algorithm might not halt at all on some input instances, or it might halt with an incorrect answer. Contrary to what you might expect, incorrect algorithms can sometimes be useful, if we can control their error rate.

Practical application of algorithms

- The Human Genome Project has made great progress toward the goals of identifying all the 100,000 genes in human DNA, determining the sequences of the 3 billion chemical base pairs that make up human DNA, storing this information in databases, and developing tools for data analysis. Each of these steps requires sophisticated algorithms.
- The Internet enables people all around the world to quickly access and retrieve large amounts of information. With the aid of clever algorithms, sites on the Internet are able to manage and manipulate this large volume of data. Examples of problems that make essential use of algorithms include finding good routes on which the data will travel

- Electronic commerce enables goods and services to be negotiated and exchanged electronically, and it depends on the privacy of personal information such as credit card numbers, passwords, and bank statements. The core technologies used in electronic commerce include public-key cryptography and digital signatures

- Manufacturing and other commercial enterprises often need to allocate scarce resources in the most beneficial way. An oil company may wish to know where to place its wells in order to maximize its expected profit. A political candidate may want to determine where to spend money buying campaign advertising in order to maximize the chances of winning an election. An airline may wish to assign crews to flights in the least expensive way possible, making sure that each flight is covered and that government regulations regarding crew scheduling are met. An Internet service provider may wish to determine where to place additional resources in order to serve its customers

Temporal and Spatial Limitations

Suppose computers were infinitely fast and computer memory was free. Would you have any reason to study algorithms? The answer is yes, if for no other reason than that you would still like to demonstrate that your solution method terminates and does so with the correct answer.

If computers were infinitely fast, any correct method for solving a problem would do. You would probably want your implementation to be within the bounds of good software engineering practice (for example, your implementation should be well designed and documented), but you would most often use whichever method was the easiest to implement.

Of course, computers may be fast, but they are not infinitely fast. And memory may be inexpensive, but it is not free. Computing time is therefore a bounded resource, and so is space in memory. You should use these resources wisely, and algorithms that are efficient in terms of time or space will help you do so.

Algorithm Efficiency

The first, known as insertion sort, takes time roughly equal to $c1n^2$ to sort n items, where $c1$ is a constant that does not depend on n. That is, it takes time roughly proportional to n^2. The second, merge sort, takes time roughly equal to $c2\ n \log n$, where $\log n$ stands for $\log_2 n$ and $c2$ is another constant that also does not depend on n. Insertion sort typically has a smaller constant factor than merge sort, so that $c1 < c2$. We shall see that the constant factors can have far less of an impact on the running time than the dependence on the input size n.

For a concrete example, let us pit a faster computer (computer A) running insertion sort against a slower computer (computer B) running merge sort. They each must sort an array of 10 million numbers. (Although 10 million numbers might seem like a lot, if the numbers are eight-byte integers, then the input occupies about 80 megabytes, which fits in the memory of even an inexpensive laptop computer many times over.) Suppose that computer A executes 10 billion instructions per second (faster than any single sequential computer at the time of this writing) and computer B executes only 10 million instructions per second, so that computer A is 1000 times

faster than computer B in raw computing power. To make the difference even more dramatic, suppose that the world's craftiest programmer codes insertion sort in machine language for computer A, and the resulting code requires $2n^2$ instructions to sort n numbers. Suppose further that just an average programmer implements merge sort, using a high-level language with an inefficient compiler, with the resulting code taking $50\, n \log n$ instructions. To sort 10 million numbers, computer A takes

$$\frac{2(10^7)^2\ inst}{10^{10}\ inst/sec} = \text{ 20,000 seconds (more than 5.5 hours) ;}$$

while computer B takes

$$\frac{50(10^7)\log_2 10^7\ inst}{10^7\ inst/sec} \approx 1163\ seconds\ (less\ than\ 20\ minutes)\ :$$

By using an algorithm whose running time grows more slowly, even with a poor compiler, computer B runs more than 17 times faster than computer A! The advantage of merge sort is even more pronounced when we sort 100 million numbers: where insertion sort takes more than 23 days, merge sort takes under four hours. In general, as the problem size increases, so does the relative advantage of merge sort.

Algorithms in the contemporary context

The example above shows that we should consider algorithms, like computer hardware, as a technology. Total system performance depends on choosing efficient algorithms as much as on choosing fast hardware. Just as rapid advances are being made in other computer technologies, they are being made in algorithms as well.

You might wonder whether algorithms are truly that important on contemporary computers in light of other advanced technologies, such as

- advanced computer architectures and fabrication technologies,
- easy-to-use, intuitive, graphical user interfaces (GUIs),
- object-oriented systems,
- integrated Web technologies, and
- fast networking, both wired and wireless.

The answer is yes. Although some applications do not explicitly require algorithmic content at the application level (such as some simple, Web-based applications), many do. For example, consider a Web-based service that determines how to travel

from one location to another. Its implementation would rely on fast hardware, a graphical user interface, wide-area networking, and also possibly on object orientation. However, it would also require algorithms for certain operations, such as finding routes (probably using a shortest-path algorithm), rendering maps, and interpolating addresses.

Moreover, even an application that does not require algorithmic content at the application level relies heavily upon algorithms. Does the application rely on fast hardware? The hardware design used algorithms. Does the application rely on graphical user interfaces? The design of any GUI relies on algorithms. Does the application rely on networking? Routing in networks relies heavily on algorithms. Was the application written in a language other than machine code? Then it was processed by a compiler, interpreter, or assembler, all of which make extensive use

Furthermore, with the ever-increasing capacities of computers, we use them to solve larger problems than ever before. As we saw in the above comparison between insertion sort and merge sort, it is at larger problem sizes that the differences in efficiency between algorithms become particularly prominent. Having a solid base of algorithmic knowledge and technique is one characteristic that separates the truly skilled programmers from the novices. With modern computing technology, you can accomplish some tasks without knowing much about algorithms, but with a good background in algorithms, you can do much, much more.

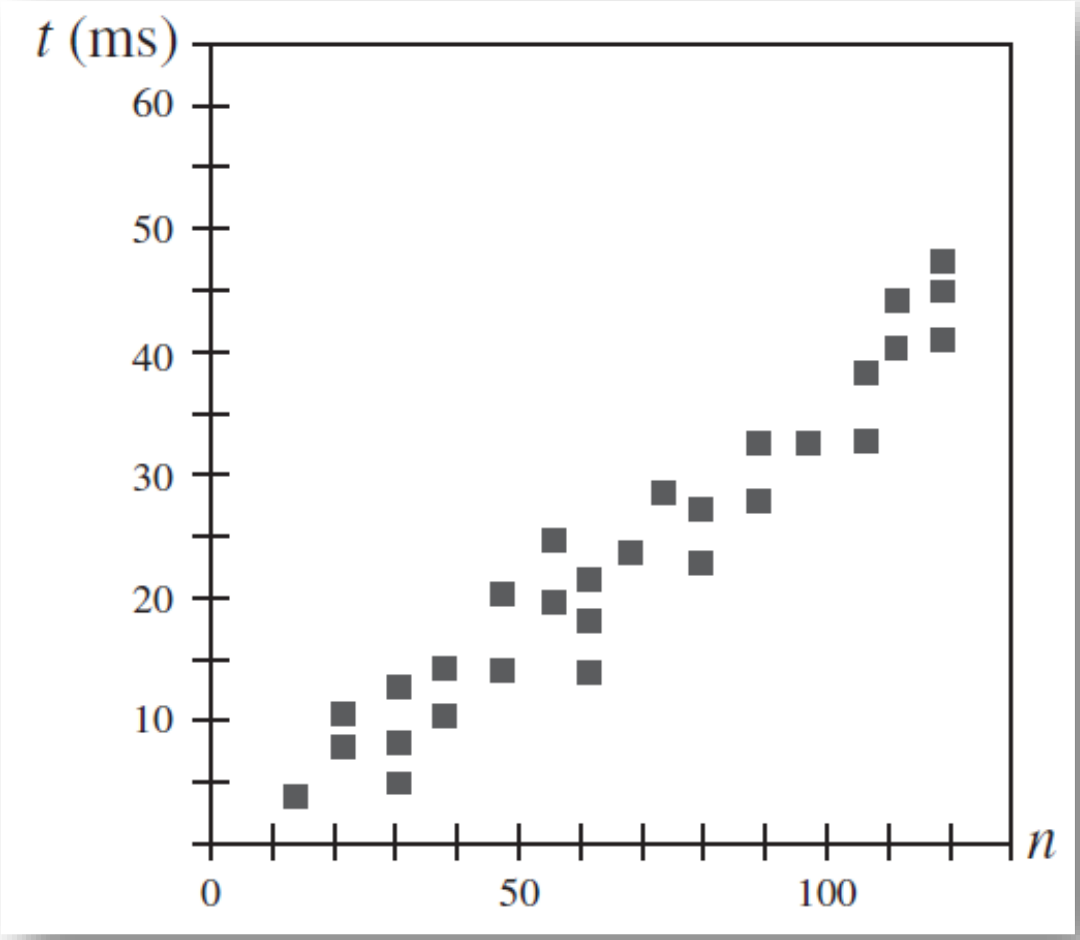
Lesson 2: Recurrence relations, Asymptotic Notation and Memory Complexity analysis

Learning Outcomes

- LO 1.2.1 Simplify the recurrence relations of functions.
- LO 1.2.1 Determine class complexity of problems together with their upper bound and lower bound.
- LO 1.2.1 Perform memory complexity analysis.

Asymptotic Notation for Analyzing Algorithms

The running time of an algorithm or data structure operation typically depends on a number of factors, so what should be the proper way of measuring it? If an algorithm has been implemented, we can study its running time by executing it on various test inputs and recording the actual time spent in each execution. Such measurements can be taken in an accurate manner by using system calls that are built into the language or operating system for which the algorithm is written. In general, we are interested in determining the dependency of the running time on the size of the input. In order to determine this, we can perform several experiments on many different test inputs of various sizes. We can then visualize the results of such experiments by plotting the performance of each run of the algorithm as a point with x-coordinate equal to the input size, n, and y-coordinate equal to the running time, t.



Counting Primitives

As we noted above, experimental analysis is valuable, but it has its limitations. If we wish to analyze a particular algorithm without performing experiments on its running time, we can take the following more analytic approach directly on the high-level code or pseudocode. We define a set of high-level *primitive operations* that are largely independent from the programming language used and can be identified also in the pseudocode. Primitive operations include the following:

- Assigning a value to a variable
- Calling a method
- Performing an arithmetic operation (for example, adding two numbers)
- Comparing two numbers
- Indexing into an array
- Following an object reference
- Returning from a method.

Specifically, a primitive operation corresponds to a low-level instruction with an execution time that depends on the hardware and software environment but is nevertheless constant. Instead of trying to determine the specific execution time of each primitive operation, we will simply count how many primitive operations are

executed, and use this number t as a high-level estimate of the running time of the algorithm.

This approach of simply counting primitive operations gives rise to a computational model called the ***Random Access Machine*** (RAM). This model, which should not be confused with “random access memory,” views a computer simply as a CPU connected to a bank of memory cells. Each memory cell stores a word, which can be a number, a character string, or an address—that is, the value of a base type. The term “random access” refers to the ability of the CPU to access an arbitrary memory cell with one primitive operation. To keep the model simple, we do not place any specific limits on the size of numbers that can be stored in words of memory.
We assume the CPU in the RAM model can perform any primitive operation in a constant number of steps, which do not depend on the size of the input. Thus, an accurate bound on the number of primitive operations an algorithm performs corresponds directly to the running time of that algorithm in the RAM model.

We now show how to count the number of primitive operations executed by an algorithm, using as an example algorithm arrayMax implemented in C.

```
int arrayMax(int A[], int n)
{
    int currentMax = A[0];
    for (int i=0;i<n;i++)
    {
        if (currentMax<A[i])
            currentMax=A[i];
    }
    return currentMax;
}
```

Initializing the variable currentMax to A[0] corresponds to two primitive operations (indexing into an array and assigning a value to a variable) and is executed only once at the beginning of the algorithm. Thus, it contributes two units to the count.

At the beginning of the for loop, counter i is initialized to 1. This action corresponds to executing one primitive operation (assigning a value to a variable).

Before entering the body of the for loop, condition $i < n$ is verified. This action corresponds to executing one primitive instruction (comparing two numbers). Since counter i starts at 1 and is incremented by 1 at the end of each iteration of the loop, the comparison $i < n$ is performed n times. Thus, it contributes n units to the count.

The body of the for loop is executed $n - 1$ times (for values 1, 2, . . . , n – 1 of the counter). At each iteration, A[i] is compared with currentMax (two primitive operations, indexing and comparing), A[i] is possibly assigned to currentMax (two primitive operations, indexing and assigning), and the counter i is incremented (two primitive operations, summing and assigning). Hence, at each iteration of the loop, either four or six primitive operations are performed, depending on whether A[i] ≤ currentMax or A[i] > currentMax. Therefore, the body of the loop contributes between $4(n - 1)$ and $6(n - 1)$ units to the count.

Returning the value of variable currentMax corresponds to one primitive operation, and is executed only once.

To summarize, the number of primitive operations t(n) executed by algorithm arrayMax is at least

$$2 + 1 + n + 4(n - 1) + 1 = 5n$$

and at most

$$2 + 1 + n + 6(n - 1) + 1 = 7n - 2$$

The best case (t(n) = 5n) occurs when A[0] is the maximum element, so that variable currentMax is never reassigned. The worst case (t(n) = 7n − 2) occurs when the elements are sorted in increasing order, so that variable currentMax is reassigned at each iteration of the for loop.

Like the arrayMax method, an algorithm may run faster on some inputs than it does on others. In such cases we may wish to express the running time of such an algorithm as an average taken over all possible inputs. Although such an average case analysis would often be valuable, it is typically quite challenging. It requires us to define a probability distribution on the set of inputs, which is typically a difficult task.

An average-case analysis also typically requires that we calculate expected running times based on a given input distribution. Such an analysis often requires heavy mathematics and probability theory.

Therefore, except for experimental studies or the analysis of algorithms that are themselves randomized, typically characterize running times in terms of the worst case. We say, for example, that algorithm arrayMax executes t(n) = 7n−2 primitive operations in the worst case, meaning that the maximum number of primitive operations executed by the algorithm, taken over all inputs of size n, is 7n − 2.

Analyzing Recursive Algorithms

Iteration is not the only interesting way of solving a problem. Another useful technique, which is employed by many algorithms, is to use *recursion*. In this technique, we define a procedure *P* that is allowed to make calls to itself as a subroutine, provided those calls to *P* are for solving subproblems of smaller size. The subroutine calls to *P* on smaller instances are called “recursive calls.” A recursive procedure should always define a *base case*, which is small enough that the algorithm can solve it directly without using recursion. We give a recursive solution to the array maximum problem in Algorithm below implemented in C.

```
int recursiveMax(int A[], int n)
{
    // if n = 0 means whole array has been traversed
    if (n == 1)
        return A[0];
    return max(A[n-1], recursiveMax(A, n-1));
}
```

We give a recursive solution to the array maximum problem in Algorithm 1.4. This algorithm first checks if the array contains just a single item, which in this case must

be the maximum; hence, in this simple base case we can immediately solve the problem. Otherwise, the algorithm recursively computes the maximum of the first n − 1 elements in the array and then returns the maximum of this value and the last element in the array.

As with this example, recursive algorithms are often quite elegant. Analyzing the running time of a recursive algorithm takes a bit of additional work, however. In particular, to analyze such a running time, we use a recurrence equation, which defines mathematical statements that the running time of a recursive algorithm must satisfy. We introduce a function T(n) that denotes the running time of the algorithm on an input of size n, and we write equations that T(n) must satisfy. For example, we can characterize the running time, T(n), of the recursiveMax algorithm as

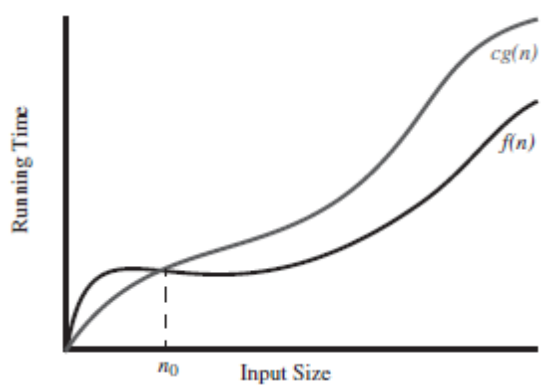
$$T(n) = \begin{cases} 3, & if\ n = 1 \\ T(n-1) + 7, & otherwise \end{cases}$$

assuming that we count each comparison, array reference, recursive call, max calculation, or return as a single primitive operation. Ideally, we would like to characterize a recurrence equation like that above in *closed form*, where no references to the function *T* appear on the righthand side. For the recursiveMax algorithm, it isn't too hard to see that a closed form would be

$$T(n) = 7(n-1) + 3 = 7n - 4$$

The "Big-Oh" Notation

Let *f*(*n*) and *g*(*n*) be functions mapping nonnegative integers to real numbers. We say that *f*(*n*) is *O*(*g*(*n*)) if there is a real constant *c* > 0 and an integer constant *n*0 ≥ 1 such that *f*(*n*) ≤ *cg*(*n*) for every integer *n* ≥ *n*0. This definition is often pronounced as "*f*(*n*) is *big-Oh* of *g*(*n*)" or "*f*(*n*) is *order g*(*n*)."



The big-Oh notation allows us to say that a function of *n* is "less than or equal to" another function (by the inequality "≤" in the definition), up to a constant factor (by

the constant *c* in the definition) and in the *asymptotic* sense as *n* grows toward infinity (by the statement "$n \geq n0$" in the definition)

Let *d(n), e(n)*, *f(n)*, and *g(n)* be functions mapping nonnegative integers to nonnegative reals.

1. If *d(n)* is *O(f(n))*, then *ad(n)* is *O(f(n))*, for any constant $a > 0$.
2. If *d(n)* is *O(f(n))* and *e(n)* is *O(g(n))*, then *d(n)+e(n)* is *O(f(n)+g(n))*.
3. If *d(n)* is *O(f(n))* and *e(n)* is *O(g(n))*, then *d(n)e(n)* is *O(f(n)g(n)).*
4. If d(n) is O(f(n)) and f(n) is O(g(n)), then d(n) is O(g(n)).
5. If *f(n)* is a polynomial of degree *d* (that is, $f(n) = a_0 + a_1 n + \cdot \cdot \cdot + a_d n^d$), then *f(n)* is $O(n^d)$.
6. n^x is $O(a^n)$ for any fixed $x > 0$ and $a > 1$.
7. $\log n^x$ is $O(\log n)$ for any fixed $x > 0$.
8. $\log^x n$ is O(ny) for any fixed constants x > 0 and y > 0.

Example:

$$2n^3 + 4n^2 \log n \;\; is\ O(n^3)$$

Big-Omega and Big-Theta

Just as the big-Oh notation provides an asymptotic way of saying that a function is "less than or equal to" another function, there are other notations that provide asymptotic ways of making other types of comparisons.

Let *f(n)* and *g(n)* be functions mapping integers to real numbers. We say that *f(n)* is *Ω(g(n))* (pronounced "*f(n)* is big-Omega of *g(n)*") if *g(n)* is *O(f(n));* that is, there is a real constant c > 0 and an integer constant n0 ≥ 1 such that $f(n) \geq cg(n)$, for n ≥ n0. This definition allows us to say asymptotically that one function is greater than or equal to another, up to a constant factor. Likewise, we say that *f(n)* is *Θ(g(n))* (pronounced "*f(n)* is big-Theta of *g(n)*") if *f(n)* is *O(g(n))* and *f(n)* is *Ω(g(n));* that is, there are real constants c> 0 and c> 0, and an integer constant n0 ≥ 1 such that $cg(n) \leq f(n) \leq cg(n)$, for n ≥ n0.

The big-Theta allows us to say that two functions are asymptotically equal, up to a constant factor. We consider some examples of these notations below.

Example:

$$3 \log n \;+\; \log \log n \text{ is } \Omega(\log n).$$

Since:

$$3 \log n \;+\; \log \log n \;\geq\; 3 \log n, \text{for } n \;\geq\; 2..$$

Words of Caution

A few words of caution about asymptotic notation are in order at this point. First, note that the use of the big-Oh and related notations can be somewhat misleading should the constant factors they "hide" be very large. For example, while it is true that the function $10^{100}n$ is *Θ(n)*, if this is the running time of an algorithm being compared to one whose running time is *10n log n*, we should prefer the *Θ(n log n)*-time algorithm, even though the linear-time algorithm is asymptotically faster. This preference is because the constant factor, 10^{100}, which is called "one googol," is believed by many astronomers to be an upper bound on the number of atoms in the

observable universe. So we are unlikely to ever have a real-world problem that has this number as its input size. Thus, even when using the big-Oh notation, we should at least be somewhat mindful of the constant factors and lower order terms we are " hiding."

The above observation raises the issue of what constitutes a "fast" algorithm. Generally speaking, any algorithm running in $O(n \log n)$ time (with a reasonable constant factor) should be considered efficient. Even an $O(n^2)$-time method may be fast enough in some contexts—that is, when n is small. But an algorithm running in $\Theta(2^n)$ time should never be considered efficient. This fact is illustrated by a famous story about the inventor of the game of chess. He asked only that his king pay him 1 grain of rice for the first square on the board, 2 grains for the second, 4 grains for the third, 8 for the fourth, and so on. But try to imagine the sight of 2^{64} grains stacked on the last square! In fact, this number cannot even be represented as a standard long integer in most programming languages.

Therefore, if we must draw a line between efficient and inefficient algorithms, it is natural to make this distinction be that between those algorithms running in polynomial time and those requiring exponential time. That is, make the distinction between algorithms with a running time that is $O(n^k)$, for some constant $k \geq 1$, and those with a running time that is $\Theta(c^n)$, for some constant $c > 1$. Like so many notions we have discussed in this section, this too should be taken with a "grain of salt," for an algorithm running in $\Theta(n^{100})$ time should probably not be considered " efficient." Even so, the distinction between polynomial-time and exponential-time algorithms is considered a robust

Importance of Asymptotics

Asymptotic notation has many important benefits, which might not be immediately obvious. Specifically, we illustrate one important aspect of the asymptotic viewpoint in Table 1.1. This table explores the maximum size allowed for an input instance for various running times to be solved in 1 second, 1 minute, and 1 hour, assuming each operation can be processed in 1 microsecond (1 μs). It also shows the importance of algorithm design, because an algorithm with an asymptotically slow running time (for example, one that is $O(n^2)$) is beaten in the long run by an algorithm with an asymptotically faster running time (for example, one that is $O(n \log n)$), even if the constant factor for the faster algorithm is worse.

Table 1: Maximum size of a problem that can be solved in one second, one minute, and one hour, for various running times measured in microseconds.

Running Time	**Maximum Problem Size (n)**		
Maximum Problem Size (n)	**1 second**	**1 minute**	**1 hour**

400n	2,500	150,000	9,000,000
20n(log n)	4,096	166,666	7,826,087
$2n^2$	707	5,477	42,426
n^4	31	88	244
2^n	19	25	31

In Table 1.2, we illustrate the difference in the growth rate of the functions

n	$\log n$	$\log^2 n$	$\sqrt{n}$	$n \log n$	n^2	n^3	2^n
4	2	4	2	8	16	64	16
16	4	16	4	64	256	4,096	65,536
64	6	36	8	384	4,096	262,144	1.84×10^{19}
256	8	64	16	2,048	65,536	16,777,216	1.15×10^{77}
1,024	10	100	32	10,240	1,048,576	1.07×10^{9}	1.79×10^{308}
4,096	12	144	64	49,152	16,777,216	6.87×10^{10}	10^{1233}
16,384	14	196	128	229,376	268,435,456	4.4×10^{12}	10^{4932}
65,536	16	256	256	1,048,576	4.29×10^{9}	2.81×10^{14}	10^{19728}
262,144	18	324	512	4,718,592	6.87×10^{10}	1.8×10^{16}	10^{78913}

Mathematical tools for analysis

Summations

A notation that appears again and again in the analysis of data structures and algorithms is the *summation*, which is defined as:

$$\sum_{i=a}^{b} f(i) = f(a) + f(a+1) + f(a+2) + \cdots + f(b).$$

Summations arise in data structure and algorithm analysis because the running times of loops naturally give rise to summations. For example, a summation that often arises in data structure and algorithm analysis is the geometric summation.

For any integer n ≥ 0 and any real number 0 < a ≠ 1, consider

$$\sum_{i=0}^{n-1} a^i = 1 + a + a^2 + a^3 \ldots a^{n-1}$$

Summations as shown above are called geometric summations, because each term is geometrically larger than the previous one if a > 1. That is, the terms in such a geometric summation exhibit exponential growth.

For example, everyone working in computing should know that

$$1 + 2 + 3 + 4 + 8 + \cdots 2^{n-1} = 2^n - 1$$

Another summation that arises in several contexts is

$$\sum_{i=1}^{n} i = 1 + 2 + 3 + \cdots + (n - 2) + (n - 1) + n$$

This summation often arises in the analysis of loops in cases where the number of operations performed inside the loop increases by a fixed, constant amount with each iteration.

Logarithms and Exponents

One of the interesting and sometimes even surprising aspects of the analysis of data structures and algorithms is the ubiquitous presence of logarithms and exponents, where we say:

$$\log_b a = c \ if \ a = b^c$$

There are a number of important rules for logarithms and exponents, including the following:

1. $\log_b ac = \log_b a + \log_b c$
2. $\log_b a/c = \log_b a - \log_b c$
3. $\log_b a^c = c\log_b a$
4. $\log_b a = \log_c a \ / \ \log_c b$
5. $\mathrm{b}^{\log_c a} = \mathrm{a}^{\log_c b}$
6. $(b^a)^c = b^{ac}$
7. $b^a b^c = b^{a+c}$
8. $b^a/b^c = b^{a-c}$

The Floor and Ceiling Functions

One additional comment concerning logarithms is in order. The value of a logarithm is typically not an integer, yet the running time of an algorithm is typically expressed by means of an integer quantity, such as the number of operations performed. Thus, an algorithm analysis may sometimes involve the use of the so-called "floor" and " ceiling" functions, which are defined respectively as follows:

$\lfloor x \rfloor$ = the largest integer less than or equal to x

$\lceil x \rceil$ = the smallest integer greater than or equal to x.

Simple Justification Techniques

- **By Example** - "Every element x in a set S has property P." To justify that such a claim is false, we need to only produce a particular x from S that does not have property P. Such an instance is called a counterexample.
- **Contrapositives and Contradiction** - The two primary such methods are the use of the *contrapositive* and the *contradiction*. To justify the statement "if *p* is true, then *q* is true," we instead establish that "if *q* is not true, then *p* is not true."
- **Induction -** This technique amounts to showing that, for any particular n ≥ 1, there is a finite sequence of implications that starts with something known to be rue and ultimately leads to showing that q(n) is true. Then we justify that the inductive "step" is true for n > k, namely, we show "if q(i) is true for i < n, then q(n) is true." The combination of these two pieces completes the proof by induction.

Basic Probability

The most basic is that any statement about a probability is defined upon a sample space S, which is defined as the set of all possible outcomes from some experiment.

A probability space is a sample space S together with a probability function, Pr, that maps subsets of S to real numbers in the interval [0, 1]. It captures mathematically the notion of the probability of certain "events" occurring. Formally, each subset A of S is called an event, and the probability function Pr is assumed to possess the following basic properties with respect to events defined from S:

1. $\Pr(\emptyset) = 0$.
2. $\Pr(S) = 1$.
3. $0 \leq \Pr(A) \leq 1$, for any $A \subseteq S$.
4. If $A,B \subseteq S$ and $A \cap B = \emptyset$, then $\Pr(A \cup B) = \Pr(A) + \Pr(B)$

Independence

Two events A and B are independent if $\Pr(A \cap B) = \Pr(A) \cdot \Pr(B)$

A collection of events $\{A1, A2, \ldots, An\}$ is *mutually independent* if $\Pr(Ai1 \cap Ai2 \cap \ldots \cap Aik) = \Pr(Ai1)\Pr(Ai2)\ldots\Pr(Aik)$,

Conditional Probability

The conditional probability that an event A occurs, given an event B, is denoted as Pr(A|B), and is defined as

$$Pr(A|B) = \frac{\Pr(A \cap B)}{\Pr(B)}$$

Recurrence relations

A recurrence is an equation or an inequality that describes a function in terms of its value on smaller inputs.

$$T(n) = aT\left(\frac{n}{b}\right) + f(n)$$

Memory Complexity analysis

This is the amount of memory needed in the execution of the algorithm. Usually, this can easily be determined by examining the data structures of the algorithm. Suppose: Matrix reverse that would require another matrix for storage would have the following:

$$s(n) = O(n) + O(n)$$